

Micro-Transformer: A From-Scratch Autograd Engine and a Gradient-Flow Ablation of Normalization & Residual Connections

Manikanta Harsha Nadimpalli (nmharsha) ■ Luis Blanco Hoyos (blanco1) ■ CS229, General Machine Learning

The Question

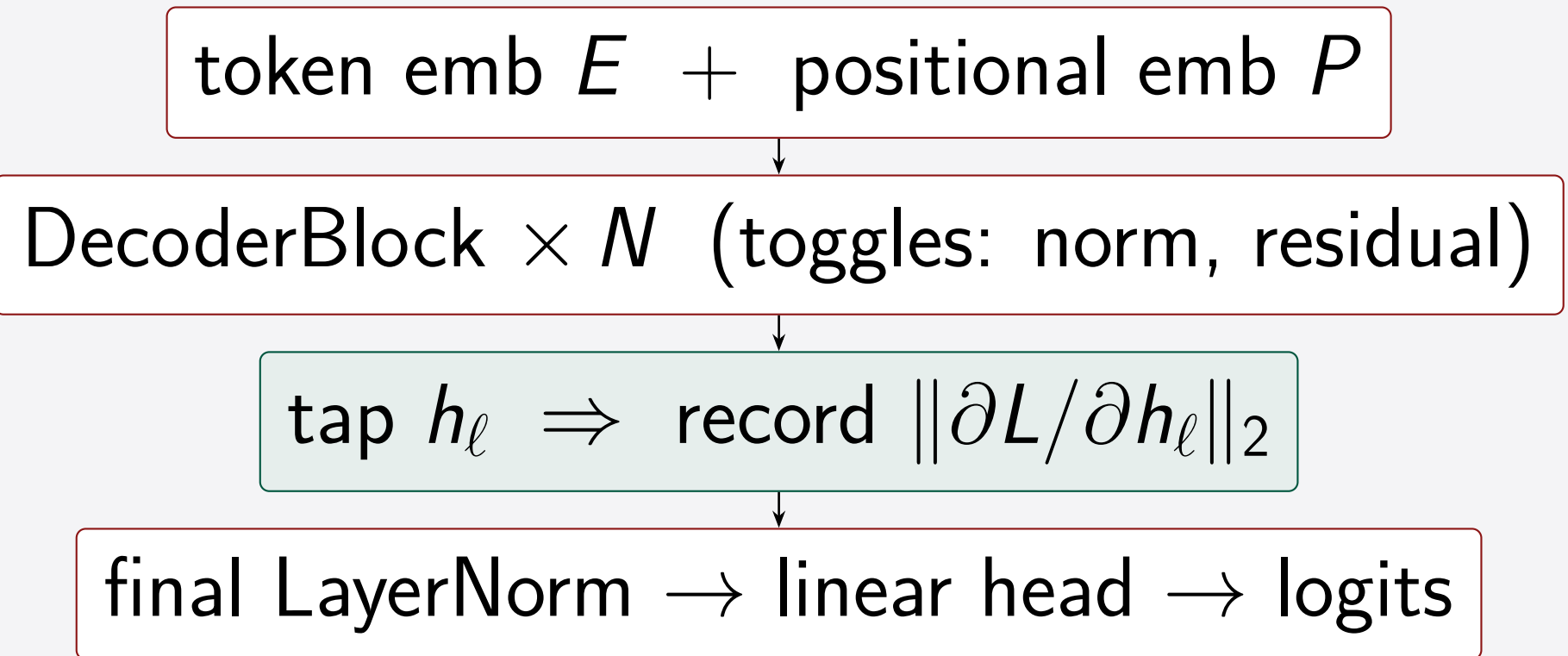
Residual (skip) connections and LayerNorm are the scaffolding of every transformer, usually treated as fixed. We make them the **object of study**:

How do LayerNorm placement (pre / post / none) and residual connections shape the gradient that reaches the early layers during training?

Input: a window of character indices $x_{1:T}$. **Output:** next-char distribution $\hat{p}(\cdot \mid x_{1:t})$ at every t . We watch not just the loss, but the intermediate gradient $\partial L / \partial h_\ell$ at each block.

Build It Ourselves → Measure Directly

A reverse-mode autograd engine in pure NumPy gives *exact, low-overhead* access to per-layer gradients — no framework hooks to reverse-engineer.



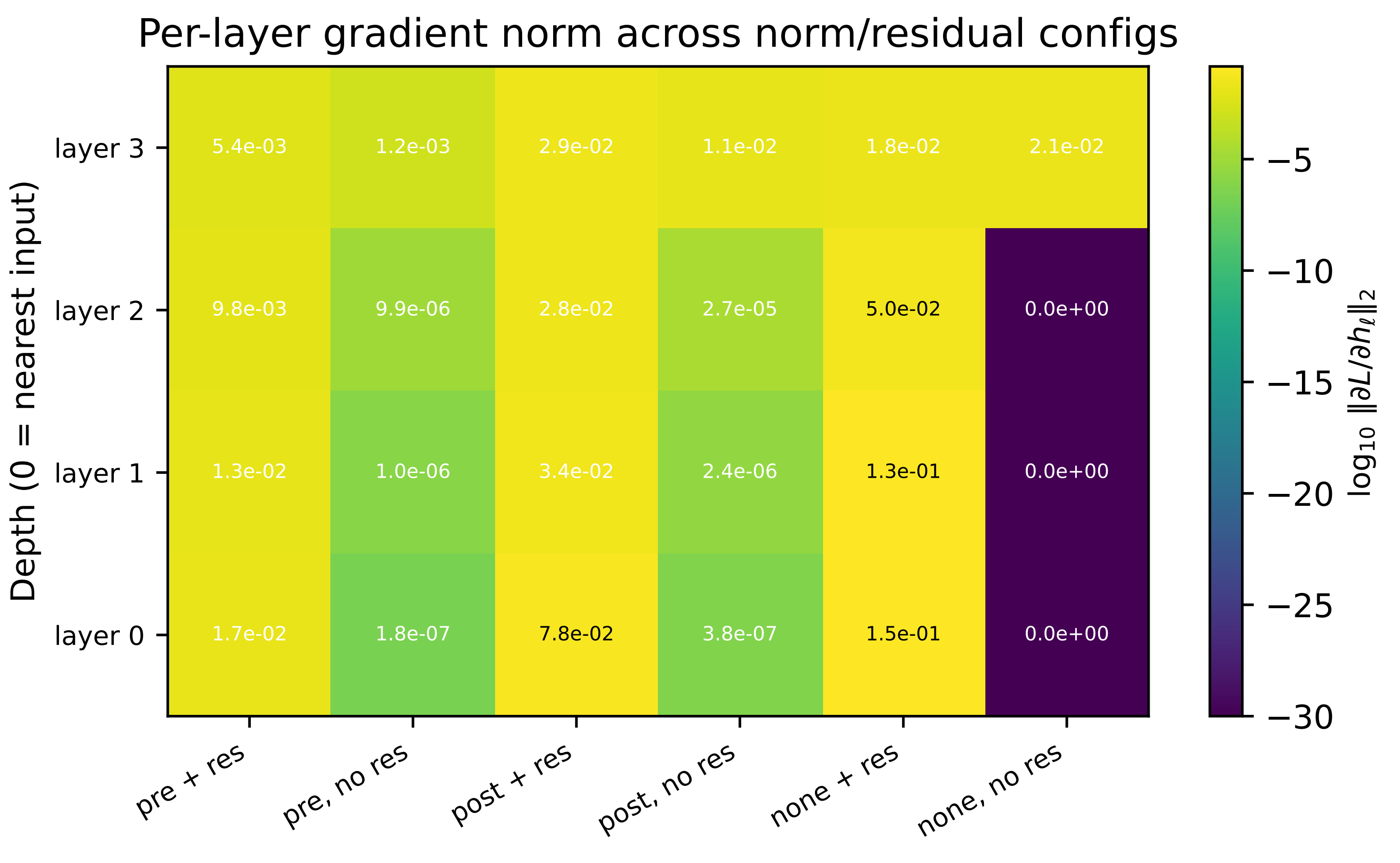
Validated to **machine precision** ($\sim 10^{-10}$ – 10^{-16}) at every layer vs. finite differences and a PyTorch oracle. **164 passing tests**. PyTorch is used *only* as a test oracle, never in the model.

The Six-Cell Grid

3 norm placements $\times$ 2 residual settings, trained *identically* (same seed, data, AdamW, 2000 steps) so the only variable is the toggle.

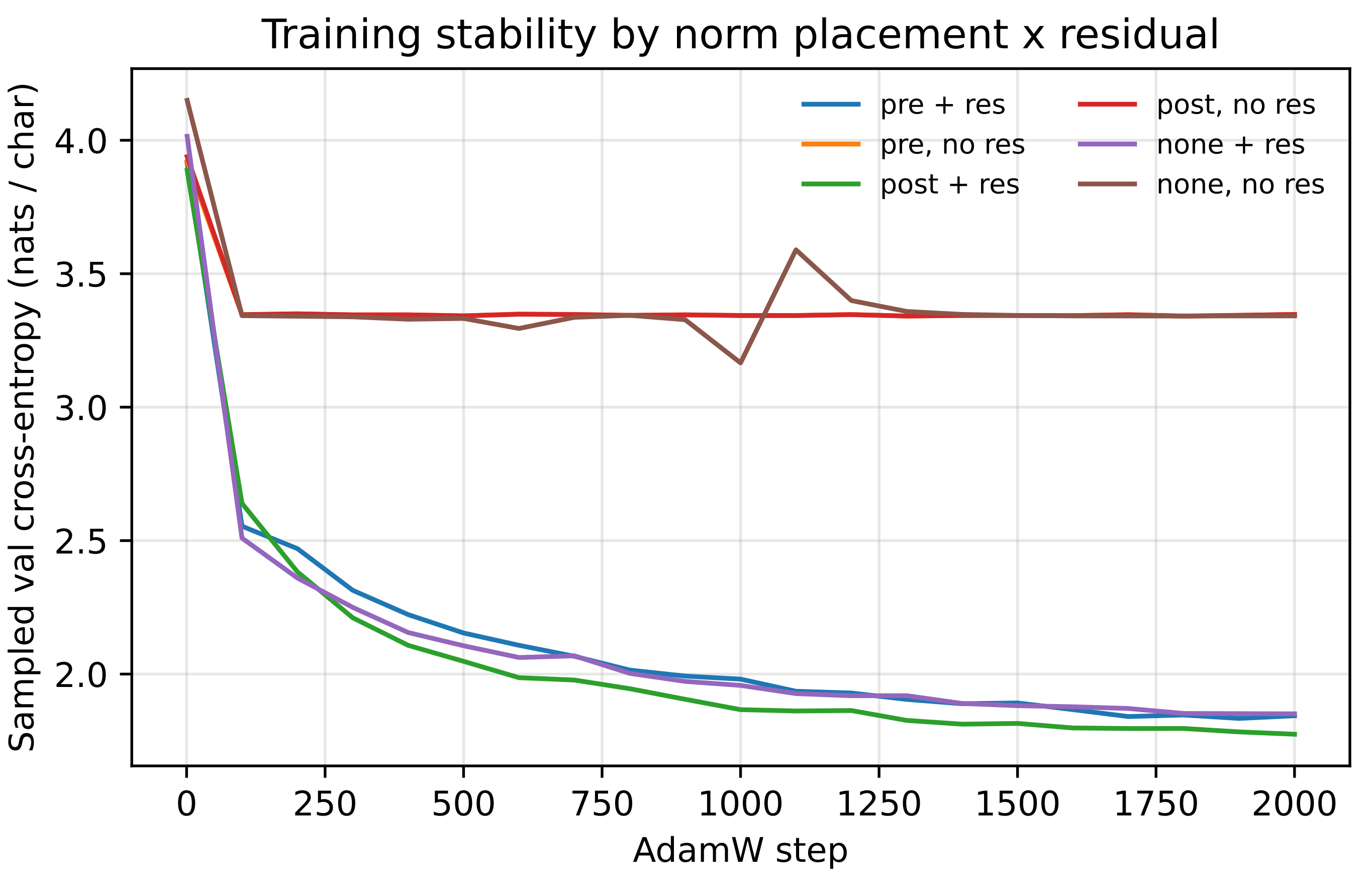
	pre	post	none
residual on	$x + f(\text{LN } x)$	$\text{LN}(x + fx)$	$x + fx$
residual off	$f(\text{LN } x)$	$\text{LN}(fx)$	fx

Hero Result: Per-Layer Gradient Norm



End-of-training gradient norm by depth ($\log_{10}$). **Residual-on** (left 3 cols) stays $O(10^{-2})$ at every layer; **residual-off** vanishes toward the input — none/no-res is *exactly zero* in layers 0–2, so those layers never learn.

Training Stability



The three residual-on configs converge together near **1.8 nats**; the three residual-off configs plateau near **3.3**.

Engine Validation Ladder

Component	Parity
Every primitive op	finite diff, atol 1e-7
Attention backward	torch / closed form, 10^{-10}
LayerNorm	torch.nn.LayerNorm, 5×10^{-16}
Decoder block	torch reconstruction, 10^{-10}
AdamW (25 steps)	torch.optim.AdamW, 10^{-10}

Results: Residual Decides, Not Norm

	Norm Res	Val CE	Decay ρ
post on	1.775	0.71	
pre on	1.844	0.68	
none on	1.851	0.50	
pre off	3.347	18.6	
post off	3.347	30.7	
none off	3.342	2746	

Decay rate $\rho = (\|g_{\text{last}}\| / \|g_{\text{first}}\|)^{1/(L-1)}$: $\rho \approx 1$ preserves the gradient, $\rho \gg 1$ vanishes it. Baselines for scale: uniform 4.17, bigram floor 2.48, char-MLP 2.28 nats.

The Takeaway

At depth 4, the **residual connection is the decisive factor** — not normalization placement.

With a skip, the gradient gets an identity path to the input and stays $O(1)$ across depth; **all** norm placements then train and beat the bigram floor. **Without** it, the gradient must cross every sublayer Jacobian and decays geometrically — early layers go untrainable regardless of norm. Pre- vs. post-norm is a *second-order* effect at this scale (the known pre-norm advantage emerges deeper, without an adaptive optimizer).

Scope, Limits & Next Steps

Single seed/cell, single-head, one corpus, depth 4 — so post-vs-pre is “comparable,” not a ranking. **Next:** multi-seed error bars, greater depth (where pre/post diverges), multi-head, and Jacobian singular-value spectra.

Contributions & Code

Harsha: data, baselines, scalar+tensor autograd, attention/LayerNorm/ block/transformer in-engine, AdamW, instrumentation, ablation, writeups.

Luis: closed-form attention-backward derivation + PyTorch oracle + parity tests (the independent spec our engine is verified against).

github.com/harshanm65/jacobigrad